UNIVERSIDAD NACIONAL DEL ALTIPLANO • PUNO
Escuela Profesional de Ingeniería de Sistemas

INFORME DE PRÁCTICA DE LABORATORIO • PRÁCTICA N.° 07

Tablas Hash

Implementación, colisiones y miniestudio experimental • Python y C++

CURSO	Algoritmos y Estructuras de Datos — SIS210
SEMANA	7
LENGUAJE	Python 3.11+ / C++17
DURACIÓN	3 horas académicas
DOCENTE	Dr. Aldo Hernán Zanabria Gálvez
ORCID / RENACYT	0000-0003-3314-8768 | P0020608

Puno — Perú — 2026

CONTENIDO

I. MARCO TEÓRICO . . . 2
Objetivo de la práctica . . . 2
¿Qué problema resuelve una tabla hash? . . . 2
Funciones hash . . . 3
Colisiones y factor de carga . . . 3
Resolución por encadenamiento . . . 4
Direccionamiento abierto . . . 4
Rehashing y comparación de estrategias . . . 4
Aplicaciones y relación con la práctica . . . 5
II. DESARROLLO DE ACTIVIDADES . . . 6
Actividad 1: Python — Funciones hash y análisis de distribución . . . 6
Actividad 2: Python — TablaHashChaining con encadenamiento completo . . . 7
Actividad 3: Python — TablaHashOpenAddressing con sondeo lineal y marcadores . . . 10
Actividad 4: Python — Miniestudio experimental: factor de carga vs colisiones . . . 12
Complemento: Python — Tablas propias vs dict . . . 14
Actividad 5: C++ — Hash personalizado y unordered_map . . . 15
Actividad 6: Python — Aplicación real: índice invertido con tabla hash . . . 18
III. PREGUNTAS DE ANÁLISIS . . . 21

I. MARCO TEÓRICO

Objetivo de la práctica

Objetivo general. Implementar, analizar experimentalmente y comparar distintas estrategias de tablas hash, entendiendo cómo el factor de carga y la resolución de colisiones determinan su rendimiento.

Objetivos específicos.

- Implementar la función hash por módulo y la hash polinomial, y medir su distribución con el coeficiente de variación (Actividad 1).
- Construir una tabla hash con **encadenamiento**, con inserción, búsqueda, eliminación y *rehashing* automático (Actividad 2).
- Construir una tabla hash con **direccionamiento abierto** y sondeo lineal, resolviendo el problema de la eliminación con marcadores `DELETED` (Actividad 3).
- Medir cómo crece el número de colisiones al aumentar el factor de carga α y contrastarlo con la teoría (Actividad 4).
- Comparar una función hash propia (FNV-1a) con `std::hash` sobre `unordered_map` en C++ (Actividad 5).
- Aplicar una tabla hash a un problema real: un índice invertido de documentos (Actividad 6).

¿Qué problema resuelve una tabla hash?

Muchos programas necesitan un **diccionario**: una colección de pares *(clave, valor)* sobre la que se pueda **insertar**, **buscar** y **eliminar** por clave. Por ejemplo, asociar un nombre de ciudad con su altitud, una palabra con los documentos donde aparece, o un identificador de usuario con su sesión.

Las estructuras clásicas obligan a elegir entre buscar rápido o insertar rápido:

Tabla 1 | Costo de las operaciones del diccionario según la estructura de datos (n elementos).

Estructura	Buscar	Insertar	Eliminar
Lista no ordenada	$O(n)$	$O(1)^*$	$O(n)$
Arreglo ordenado	$O(\log n)$	$O(n)$	$O(n)$
Árbol balanceado (AVL, rojo-negro)	$O(\log n)$	$O(\log n)$	$O(\log n)$
Tabla hash	$\boldsymbol{O(1)}$ **promedio**	$\boldsymbol{O(1)}$ **amortizado**	$\boldsymbol{O(1)}$ **promedio**

* *Insertando sin comprobar si la clave ya existe; si se comprueba, también es* $O(n)$.

> **IDEA CLAVE**
>
> Si las claves fueran enteros pequeños, bastaría un arreglo con acceso directo $A[k]$. Pero el *universo* de claves posibles U (todos los textos de 8 letras, todos los números de documento) es enorme comparado con las pocas claves realmente guardadas. Una **tabla hash** comprime ese universo en m celdas mediante una **función hash** $h : U \to \{0, 1, \ldots, m-1\}$ y guarda cada elemento en la celda $h(k)$.

El precio de esta compresión es que dos claves distintas pueden caer en la misma celda: una **colisión**. Como $|U| > m$, por el principio del palomar las colisiones son inevitables, así que toda tabla hash necesita una estrategia para resolverlas. Ese es el centro de esta práctica.

Funciones hash

Una buena función hash debe ser **determinista** (la misma clave siempre da la misma celda), **rápida** de calcular y **uniforme** (reparte las claves de forma pareja entre las m celdas). En la práctica se usan tres:

- **Módulo (enteros):** $h(k) = k \bmod m$. Conviene que m sea un número primo, lejos de una potencia de 2, para que patrones regulares en las claves no se concentren en pocas celdas.
- **Polinomial (cadenas):**

$$h(s) = \left(\sum_{i=0}^{|s|-1} s_i \, p^i \right) \bmod m$$

  donde s_i es el código del i-ésimo carácter y p una base pequeña (aquí $p = 31$). Su costo es $O(|s|)$ y depende de todos los caracteres y de su posición.
- **FNV-1a (64 bits):** se parte de un valor inicial fijo y, por cada byte b de la clave, se calcula

$$\text{hash} \leftarrow (\text{hash} \oplus b) \cdot 1099511628211 \pmod{2^{64}}$$

  Es simple, no guarda estado y procesa un byte por iteración.

¿Cómo medir si una función reparte bien? Se cuenta cuántas claves caen en cada celda y se calcula el **coeficiente de variación** $CV = \sigma / \mu$ de esas ocupaciones. Si n claves se reparten al azar entre m celdas, la ocupación de cada una sigue aproximadamente una distribución binomial con media $\mu = n/m$ y varianza $\approx n/m$, de modo que

$$CV_{\text{ideal}} \approx \frac{\sqrt{n/m}}{n/m} = \sqrt{\frac{m}{n}}.$$

Un CV cercano a ese valor indica una distribución tan buena como la del azar; uno mucho mayor delata una función que concentra las claves.

Colisiones y factor de carga

El rendimiento de una tabla hash depende sobre todo del **factor de carga**

$$\alpha = \frac{n}{m},$$

la fracción de la tabla que está ocupada (n elementos en m celdas). Con una función uniforme, el número esperado de celdas ocupadas es $m\,(1 - e^{-\alpha})$, por lo que el número esperado de colisiones al insertar n claves es

$$n - m\,(1 - e^{-\alpha}).$$

Esta es la columna «teórico» de la Actividad 4. Las colisiones aparecen mucho antes de que la tabla se llene: con m celdas, la primera suele aparecer tras insertar del orden de $\sqrt{m}$ claves (la misma idea de la «paradoja del cumpleaños»).

Hay dos familias de estrategias para resolverlas, que se describen a continuación y se ilustran en la Figura 1.

Resolución por encadenamiento

Cada celda de la tabla guarda una **lista enlazada** con todos los elementos que caen en ella. Insertar es poner un nodo al inicio de la lista ($O(1)$); buscar o eliminar recorre solo la lista de la celda $h(k)$.

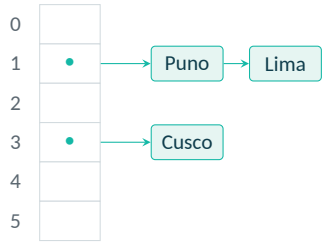
- Con hash uniforme, la longitud media de cada lista es α: una búsqueda exitosa cuesta $\approx 1 + \alpha/2$ comparaciones y una fallida $\approx 1 + \alpha$.
- Funciona incluso con $\alpha > 1$ (listas más largas) y la eliminación es directa.
- Pierde localidad de caché (los nodos están dispersos en memoria) y gasta memoria extra en los punteros.
- En el peor caso, todas las claves caen en la misma celda y la búsqueda cuesta $O(n)$.

Direccionamiento abierto

Todos los elementos se guardan **dentro del propio arreglo**, sin listas. Si la celda $h(k)$ está ocupada, se prueba otra según una secuencia de sondeo. En el **sondeo lineal**:

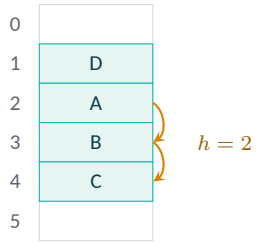
$$h(k, j) = \big(h(k) + j\big) \bmod m, \qquad j = 0, 1, 2, \ldots$$

- Requiere $\alpha < 1$: no puede guardar más elementos que celdas.
- **Clustering primario:** las celdas ocupadas forman bloques contiguos que crecen cada vez que una clave cae en ellos. Por eso los costos esperados se disparan al acercarse a $\alpha = 1$: una búsqueda exitosa cuesta $\approx \frac{1}{2}\big(1 + \frac{1}{1-\alpha}\big)$ y una fallida $\approx \frac{1}{2}\big(1 + \frac{1}{(1-\alpha)^2}\big)$ sondeos (Knuth, 1998).
- **Eliminación:** no se puede dejar la celda como vacía, porque se cortaría la cadena de sondeo de otras claves. Se usa un *marcador* `DELETED` («lápida») que la búsqueda atraviesa y que la inserción puede reutilizar (véase la Pregunta 1).
- Tiene excelente localidad de caché y no usa memoria extra por elemento.
- Otras secuencias de sondeo, como el **sondeo cuadrático** y el **doble hashing**, reducen el clustering (véase la Pregunta 2).



Encadenamiento

Puno y *Lima* colisionan en la celda 1



Sondeo lineal

A, B y C tienen $h = 2$ y ocupan celdas seguidas

Figura 1 | Las dos estrategias de resolución de colisiones.

Rehashing y comparación de estrategias

Cuando α supera un umbral (en esta práctica 0.75), la tabla se **redimensiona**: se crea una tabla nueva de tamaño primo mayor o igual al doble y se **reinsertan** todos los elementos, recalculando $h(k)$ con el nuevo m. Una operación de rehash cuesta $O(n)$, pero como el tamaño se duplica cada vez, el costo total de n inserciones es $O(n)$: cada inserción cuesta $O(1)$ *amortizado*.

Tabla 2 | Encadenamiento frente a direccionamiento abierto (sondeo lineal).

Aspecto	Encadenamiento	Direccionamiento abierto
Dónde guarda los datos	Listas fuera del arreglo	Dentro del arreglo
Factor de carga	Admite $\alpha > 1$	Exige $\alpha < 1$
Degradación al subir α	Suave, proporcional a α	Fuerte a partir de $\alpha \approx 0.75$
Eliminación	Directa	Necesita marcadores `DELETED`
Memoria	Punteros extra por nodo	Sin extra, pero tabla más holgada
Localidad de caché	Baja	Alta

Aplicaciones y relación con la práctica

Las tablas hash están en la base de muchas herramientas cotidianas:

- **Lenguajes de programación:** el `dict` y el `set` de Python (direccionamiento abierto con un diseño compacto, ver Pregunta 4) y el `unordered_map` de C++ (normalmente con encadenamiento por *buckets*, de ahí `bucket_count` y `bucket_size`; su `max_load_factor` por defecto es 1.0).
- **Índices de búsqueda:** un *índice invertido* asocia cada palabra con el conjunto de documentos que la contienen. Una consulta de varias palabras se resuelve intersectando esos conjuntos (Actividad 6).
- **Otros usos:** cachés, tablas de símbolos de compiladores, eliminación de duplicados, conteo de frecuencias y verificación de integridad.

Referencias.

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.
- Knuth, D. E. (1998). *The art of computer programming, Vol. 3: Sorting and searching* (2.ª ed.). Addison-Wesley.

II. DESARROLLO DE ACTIVIDADES

ACTIVIDAD 1

Python — Funciones hash y análisis de distribución

Objetivo. *Implementar y comparar la función hash por módulo y la hash polinomial, midiendo su distribución en la tabla mediante el coeficiente de variación.*

La celda inicial (imports y `random.seed(42)`) se incluye aquí porque la usan todas las actividades de Python; con la semilla fija los resultados son reproducibles.

Código — funciones hash y medición de distribución

```python
import random
import string
import math
import time
import re
from collections import defaultdict

random.seed(42)

def hash_modulo(k: int, m: int) -> int:
    return k % m


def hash_polinomial(s: str, m: int, p: int = 31) -> int:
    """Hash polinomial para strings: O(|s|)."""
    h = 0
    potencia = 1
    for c in s:
        h = (h + ord(c) * potencia) % m
        potencia = (potencia * p) % m
    return h


def medir_distribucion(claves, m, hash_fn) -> dict:
    """Cuenta cuántos elementos caen en cada celda y calcula estadísticas."""
    tabla = [0] * m
    for k in claves:
        tabla[hash_fn(k, m)] += 1
    ocupadas = sum(1 for c in tabla if c > 0)
    promedio = len(claves) / m
    varianza = sum((c - promedio) ** 2 for c in tabla) / m
    cv = math.sqrt(varianza) / promedio if promedio > 0 else 0
    return {'ocupadas': ocupadas, 'max_colisiones': max(tabla), 'cv': cv}

m = 101   # primo
n = 500

# Hash de enteros
claves_int = [random.randint(0, 1_000_000) for _ in range(n)]
stats = medir_distribucion(claves_int, m, hash_modulo)
print(f"Hash enteros (m={m}, n={n}): celdas_ocupadas={stats['ocupadas']}, "
      f"max_col={stats['max_colisiones']}, CV={stats['cv']:.3f}")

# Hash de strings
claves_str = [''.join(random.choices(string.ascii_lowercase, k=8)) for _ in range(n)]
stats_s = medir_distribucion(claves_str, m, hash_polinomial)
print(f"Hash strings (m={m}, n={n}): celdas_ocupadas={stats_s['ocupadas']}, "
```

```
        f"max_col={stats_s['max_colisiones']}, CV={stats_s['cv']:.3f}")

print(f"\nCV esperado para un reparto aleatorio ideal: {math.sqrt(m / n):.3f}")
print("(un CV cercano a ese valor indica buena distribución)")
```

Salida — Python

```
Hash enteros (m=101, n=500): celdas_ocupadas=100, max_col=10, CV=0.432
Hash strings (m=101, n=500): celdas_ocupadas=101, max_col=13, CV=0.483

CV esperado para un reparto aleatorio ideal: 0.449
(un CV cercano a ese valor indica buena distribución)
```

Tabla 3 | Distribución de las funciones hash ($m = 101$, $n = 500$).

Función hash	Celdas ocupadas (de 101)	Máx. claves en una celda	CV
Módulo (enteros)	100	10	0.432
Polinomial, $p = 31$ (strings)	101	13	0.483
Referencia: reparto aleatorio ideal	—	—	$\sqrt{m/n} \approx 0.449$

Análisis. Las dos funciones quedan muy cerca del CV de un reparto aleatorio ideal (0.449): la de módulo un poco por debajo y la polinomial un poco por encima. Con una sola muestra de 500 claves esa diferencia está dentro de la variación normal, así que no se puede afirmar que una sea mejor que la otra. Ambas ocupan casi todas las celdas, y el máximo de 10 y 13 claves en una celda es razonable cuando el promedio es $500/101 \approx 4.95$. Un CV cercano a 0 no sería lo esperado: significaría un reparto *más parejo que el azar*, algo que una función hash sobre claves aleatorias no logra.

ACTIVIDAD 2

Python — TablaHashChaining con encadenamiento completo

Objetivo. *Implementar la tabla hash con encadenamiento incluyendo insertar, buscar, eliminar y rehashing automático cuando $\alpha > 0.75$.*

Cada celda guarda una lista enlazada propia (`ListaEnlazada`, estilo Práctica 5). El código incluye también las funciones auxiliares `es_primo` y `siguiente_primo`.

Código — TablaHashChaining y prueba

```
def es_primo(x: int) -> bool:
    if x < 2:
        return False
    for i in range(2, int(x ** 0.5) + 1):
        if x % i == 0:
            return False
    return True


def siguiente_primo(n: int) -> int:
    while not es_primo(n):
        n += 1
    return n


```

```python
class Nodo:
    """Nodo de la lista enlazada."""
    def __init__(self, clave, valor):
        self.clave = clave
        self.valor = valor
        self.siguiente = None


class ListaEnlazada:
    """Lista enlazada simple de pares (clave, valor)."""
    def __init__(self):
        self.cabeza = None

    def buscar_nodo(self, clave):
        actual = self.cabeza
        while actual is not None:
            if actual.clave == clave:
                return actual
            actual = actual.siguiente
        return None

    def agregar(self, clave, valor):
        """Inserta al inicio: O(1)."""
        nodo = Nodo(clave, valor)
        nodo.siguiente = self.cabeza
        self.cabeza = nodo

    def eliminar(self, clave) -> bool:
        anterior = None
        actual = self.cabeza
        while actual is not None:
            if actual.clave == clave:
                if anterior is None:
                    self.cabeza = actual.siguiente
                else:
                    anterior.siguiente = actual.siguiente
                return True
            anterior = actual
            actual = actual.siguiente
        return False

    def nodos(self):
        """Recorre los nodos de la lista (se usa en el rehash)."""
        actual = self.cabeza
        while actual is not None:
            yield actual
            actual = actual.siguiente

class TablaHashChaining:
    """Tabla hash con encadenamiento. Rehash automático cuando alpha > 0.75."""
    UMBRAL = 0.75

    def __init__(self, m: int = 11):
        self._m = m
        self._n = 0
        self._tabla = [ListaEnlazada() for _ in range(m)]
        self._rehashes = 0

    def _hash(self, clave) -> int:
        return hash(clave) % self._m

    def insertar(self, clave, valor) -> None:
        lista = self._tabla[self._hash(clave)]
        nodo = lista.buscar_nodo(clave)
```

```
        if nodo is not None:          # la clave ya existe: se actualiza el valor
            nodo.valor = valor
            return
        lista.agregar(clave, valor)
        self._n += 1
        if self._n / self._m > self.UMBRAL:
            self._rehash()

    def buscar(self, clave):
        nodo = self._tabla[self._hash(clave)].buscar_nodo(clave)
        return nodo.valor if nodo is not None else None

    def eliminar(self, clave) -> bool:
        if self._tabla[self._hash(clave)].eliminar(clave):
            self._n -= 1
            return True
        return False

    def _rehash(self):
        vieja = self._tabla
        self._m = siguiente_primo(self._m * 2)
        self._tabla = [ListaEnlazada() for _ in range(self._m)]
        self._rehashes += 1
        for lista in vieja:
            for nodo in lista.nodos():
                self._tabla[self._hash(nodo.clave)].agregar(nodo.clave, nodo.valor)
        # _n no cambia: son los mismos elementos

    def __len__(self):
        return self._n

    @property
    def factor_carga(self):
        return self._n / self._m

# Prueba
ciudades = [('Puno', 3825), ('Juliaca', 1100), ('Arequipa', 2335), ('Lima', 154),
            ('Cusco', 3400), ('Iquitos', 106), ('Trujillo', 34), ('Huancayo', 3271)]

t = TablaHashChaining(m=7)
for ciudad, alt in ciudades:
    t.insertar(ciudad, alt)
print(f'alpha={t.factor_carga:.2f}, rehashes={t._rehashes}')

assert t.buscar('Puno') == 3825
assert t.eliminar('Lima')
assert t.buscar('Lima') is None
assert t.buscar('Cusco') == 3400
t.insertar('Puno', 3830)                # actualizar una clave existente
assert t.buscar('Puno') == 3830 and len(t) == 7
print('TablaHashChaining: OK ✓')
```

Salida — Python

```
alpha=0.47, rehashes=1
TablaHashChaining: OK ✓
```

Lectura del resultado. La tabla parte con $m = 7$. Al insertar el sexto elemento, $\alpha = 6/7 \approx 0.86 > 0.75$ y se dispara el rehash a $m = 17$ (el primo siguiente a $2 \cdot 7 = 14$). Con los 8 elementos queda $\alpha = 8/17 \approx 0.47$ y un solo rehash, tal como muestra la salida. Las pruebas con `assert` (buscar, eliminar y actualizar una clave existente) pasan sin error.

ACTIVIDAD 3

Python — TablaHashOpenAddressing con sondeo lineal y marcadores

Objetivo. *Implementar la tabla hash con direccionamiento abierto. El mayor desafío es la eliminación: usar marcadores DELETED para no romper las cadenas de sondeo.*

Usa `siguiente_primo` y la lista `ciudades` definidos en la Actividad 2. Esta versión recorre la cadena hasta una celda `VACIA` antes de insertar (para no duplicar claves) y cuenta los `DELETED` en el factor de carga.

Código — TablaHashOpenAddressing y pruebas

```
class TablaHashOpenAddressing:
    """Sondeo lineal con marcadores DELETED para eliminación correcta."""
    VACIA = object()     # celda nunca usada
    DELETED = object()   # celda eliminada (no corta la cadena de sondeo)
    UMBRAL = 0.70

    def __init__(self, m: int = 11):
        self._m = m
        self._n = 0            # elementos vivos
        self._borrados = 0     # marcadores DELETED
        self._tabla = [self.VACIA] * m
        self._rehashes = 0

    def _hash(self, clave, j: int) -> int:
        return (hash(clave) + j) % self._m     # sondeo lineal

    def insertar(self, clave, valor) -> None:
        if (self._n + self._borrados) / self._m > self.UMBRAL:
            self._rehash()

        primera_libre = None       # primera celda VACIA o DELETED que encontremos
        for j in range(self._m):
            idx = self._hash(clave, j)
            celda = self._tabla[idx]
            if celda is self.VACIA:
                if primera_libre is None:
                    primera_libre = idx
                break                              # aquí termina la cadena: la clave no existe
            if celda is self.DELETED:
                if primera_libre is None:
                    primera_libre = idx
            elif celda[0] == clave:                # la clave ya existe: se actualiza
                self._tabla[idx] = (clave, valor)
                return

        if primera_libre is None:
            raise RuntimeError('Tabla llena')
        if self._tabla[primera_libre] is self.DELETED:
            self._borrados -= 1
        self._tabla[primera_libre] = (clave, valor)
        self._n += 1

    def buscar(self, clave):
        for j in range(self._m):
            idx = self._hash(clave, j)
            celda = self._tabla[idx]
            if celda is self.VACIA:
                return None                        # cadena interrumpida → no existe
            if celda is not self.DELETED and celda[0] == clave:
                return celda[1]
        return None

```

```
    def eliminar(self, clave) -> bool:
        for j in range(self._m):
            idx = self._hash(clave, j)
            celda = self._tabla[idx]
            if celda is self.VACIA:
                return False
            if celda is not self.DELETED and celda[0] == clave:
                self._tabla[idx] = self.DELETED    # se marca, NO se deja VACIA
                self._n -= 1
                self._borrados += 1
                return True
        return False

    def _rehash(self):
        vieja = self._tabla
        self._m = siguiente_primo(max(11, 2 * self._n))   # deja la tabla cerca de alpha = 0.5
        self._tabla = [self.VACIA] * self._m
        self._n = 0
        self._borrados = 0
        self._rehashes += 1
        for celda in vieja:
            if celda is not self.VACIA and celda is not self.DELETED:
                self.insertar(celda[0], celda[1])

    def __len__(self):
        return self._n

    @property
    def factor_carga(self):
        return self._n / self._m

# Prueba 1: las ciudades de la Actividad 2
t2 = TablaHashOpenAddressing(m=13)
for ciudad, alt in ciudades:
    t2.insertar(ciudad, alt)
assert t2.buscar('Puno') == 3825
assert t2.eliminar('Juliaca')
assert t2.buscar('Juliaca') is None
assert t2.buscar('Puno') == 3825      # la cadena no se rompió
assert t2.buscar('Lima') == 154

# Prueba 2: claves que colisionan a propósito (con m=10, todas caen en la celda 0)
t3 = TablaHashOpenAddressing(m=10)
for k in [10, 20, 30, 40, 50]:
    t3.insertar(k, 'v' + str(k))
t3.eliminar(20)                        # queda un DELETED en medio de la cadena
assert t3.buscar(30) == 'v30'          # sigue encontrándose gracias al marcador
assert t3.buscar(50) == 'v50'
t3.insertar(40, 'nuevo')               # 40 ya existe: debe actualizarse, no duplicarse
assert t3.buscar(40) == 'nuevo' and len(t3) == 4

# Prueba 3: rehash con muchos elementos y eliminaciones
t4 = TablaHashOpenAddressing(m=5)
for i in range(100):
    t4.insertar(i, i * i)
for i in range(0, 100, 2):
    t4.eliminar(i)
assert all(t4.buscar(i) is None for i in range(0, 100, 2))
assert all(t4.buscar(i) == i * i for i in range(1, 100, 2))
print(f'rehashes={t4._rehashes}, elementos={len(t4)}, alpha={t4.factor_carga:.2f}')
print('TablaHashOpenAddressing: OK ✓')
```

Salida — Python

```
rehashes=8, elementos=50, alpha=0.26
TablaHashOpenAddressing: OK ✓
```

Lectura del resultado. Las tres pruebas pasan: (1) tras eliminar `'Juliaca'`, `'Puno'` se sigue encontrando; (2) con $m = 10$ las claves 10, 20, 30, 40 y 50 caen en la celda 0, y aun después de eliminar la 20, siguen encontrándose la 30 y la 50, y reinsertar la 40 la actualiza sin duplicarla; (3) con 100 inserciones y 50 eliminaciones se producen 8 rehashes y quedan 50 elementos vivos en una tabla de $m = 193$ ($\alpha = 50/193 \approx 0.26$).

ACTIVIDAD 4

Python — Miniestudio experimental: factor de carga vs colisiones

Objetivo. *Diseñar y ejecutar el miniestudio experimental midiendo el número real de colisiones para distintos factores de carga. Presentar los resultados en una tabla y analizar el umbral de degradación.*

Usa `hash_polinomial` de la Actividad 1. En encadenamiento se cuenta una colisión al insertar en una celda que ya tenía elementos; en direccionamiento abierto se cuenta cada sondeo fallido. *Con $\alpha > 1$ no caben todos los elementos, por eso N/A.*

Código — medición de colisiones reales

```python
def medir_colisiones_reales(n_elementos, m_tabla, n_rep=10):
    """Promedia, en n_rep repeticiones, las colisiones de ambos métodos."""
    total_ch = 0
    total_oa = 0
    oa_posible = n_elementos <= m_tabla     # con n > m no cabe en direccionamiento abierto

    for rep in range(n_rep):
        celdas_ch = [0] * m_tabla           # encadenamiento: basta contar cuántos hay en cada celda
        tabla_oa = [None] * m_tabla         # direccionamiento abierto

        for i in range(n_elementos):
            k = ''.join(random.choices(string.ascii_lowercase, k=8))
            idx = hash_polinomial(k, m_tabla)

            # Encadenamiento
            if celdas_ch[idx] > 0:
                total_ch += 1
            celdas_ch[idx] += 1

            # Direccionamiento abierto (sondeo lineal)
            if oa_posible:
                j = 0
                while tabla_oa[(idx + j) % m_tabla] is not None:
                    j += 1
                    total_oa += 1
                tabla_oa[(idx + j) % m_tabla] = k

    col_ch = total_ch / n_rep
    col_oa = total_oa / n_rep if oa_posible else None
    return col_ch, col_oa

configs = [(250, 1000), (500, 1000), (750, 1000), (1000, 1000), (1500, 1000), (2000, 1000)]

print(f"{'alpha':>6} {'n':>6} {'m':>6} {'Col_Chain':>10} {'Teórico':>9} {'Col_OpenAddr':>13} {'%_chain
    ↪ ':>8} {'%_open':>8}")
for n, m in configs:
```

```
    cc, co = medir_colisiones_reales(n, m, n_rep=10)
    alpha = n / m
    teorico = n - m * (1 - math.exp(-alpha))
    if co is None:
        print(f"{alpha:>6.2f} {n:>6} {m:>6} {cc:>10.1f} {teorico:>9.1f} {'N/A':>13} {cc / n * 100:>7.1f
    }% {'N/A':>8}")
    else:
        print(f"{alpha:>6.2f} {n:>6} {m:>6} {cc:>10.1f} {teorico:>9.1f} {co:>13.1f} {cc / n * 100:>7.1f
    }% {co / n * 100:>7.1f}%")
```

Salida — Python

```
alpha      n      m  Col_Chain   Teórico  Col_OpenAddr  %_chain   %_open
 0.25    250   1000       29.8      28.8          44.9    11.9%    18.0%
 0.50    500   1000      106.7     106.5         246.2    21.3%    49.2%
 0.75    750   1000      220.1     222.4        1066.7    29.3%   142.2%
 1.00   1000   1000      373.0     367.9       17996.8    37.3%  1799.7%
 1.50   1500   1000      723.1     723.1           N/A    48.2%      N/A
 2.00   2000   1000     1139.9    1135.3           N/A    57.0%      N/A
```

Tabla 4 | Resultados del miniestudio (10 repeticiones por fila). La columna *teórico* es $n - m(1 - e^{-\alpha})$.

alpha	n	m	Col. Chaining (prom.)	Chaining teórico	Col. OpenAddr. (prom.)	% Col./n (chaining)	% Col./n (open addr.)
0.25	250	1000	29.8	28.8	44.9	11.9 %	18.0 %
0.50	500	1000	106.7	106.5	246.2	21.3 %	49.2 %
0.75	750	1000	220.1	222.4	1066.7	29.3 %	142.2 %
1.00	1000	1000	373.0	367.9	17996.8	37.3 %	1799.7 %
1.50	1500	1000	723.1	723.1	N/A	48.2 %	N/A
2.00	2000	1000	1139.9	1135.3	N/A	57.0 %	N/A

El encadenamiento sigue de cerca el valor teórico (diferencias de 1 a 5 colisiones). En direccionamiento abierto el número es mayor porque mide el costo real de cada sondeo, no solo cuántas inserciones chocaron. Para comparar mejor, los mismos datos como costo por inserción:

Tabla 5 | Sondeos extra por inserción en direccionamiento abierto.

alpha	Sondeos extra por inserción (medido)	Teórico sondeo lineal $\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right) - 1$	Open addr. / Chaining
0.25	0.18	0.17	1.5×
0.50	0.49	0.50	2.3×
0.75	1.42	1.50	4.8×
1.00	18.0	diverge (≈ 20 para $m = 1000$)*	48.2×

* La tabla de la guía marca «∞» para $\alpha = 1$. La fórmula diverge, pero en una tabla finita y llena el costo es finito: el análisis clásico de Knuth da un orden de $\sqrt{\pi m/8} \approx 20$ para $m = 1000$, parecido al 18.0 medido.

PREGUNTA (Actividad 4)

¿A qué valor de α comienza a degradarse significativamente el rendimiento del sondeo lineal comparado con el encadenamiento?

Respuesta. La degradación se vuelve significativa **a partir de** $\alpha \approx 0.75$:

- Hasta $\alpha = 0.50$ el sondeo lineal cuesta menos de medio sondeo extra por inserción (0.49) y es 2.3 veces el costo del encadenamiento.
- En $\alpha = 0.75$ ya supera 1 sondeo extra por inserción (1.42) y la diferencia con el encadenamiento pasa a 4.8 veces. Es el umbral típico que la teoría de la guía marca.
- En $\alpha = 1.00$ el costo se dispara a 18 sondeos extra por inserción (48 veces el del encadenamiento).
- El encadenamiento, en cambio, crece de forma suave y proporcional a α (12 % $\rightarrow$ 37 % de colisiones por elemento) y además sigue funcionando con $\alpha > 1$.

COMPLEMENTO

Python — Tablas propias vs dict

Objetivo. *Comparar el rendimiento de las implementaciones propias contra el `dict` de Python insertando y buscando las mismas claves.*

Usa `TablaHashChaining` y `TablaHashOpenAddressing` (Actividades 2 y 3) y los imports de la celda inicial. Los tiempos dependen de la máquina.

Código — benchmark de las tres estructuras

```python
def benchmark(nombre, crear, insertar, buscar, claves):
    tabla = crear()

    t0 = time.perf_counter()
    for i, k in enumerate(claves):
        insertar(tabla, k, i)
    t_insertar = time.perf_counter() - t0

    t0 = time.perf_counter()
    for k in claves:
        buscar(tabla, k)
    t_buscar = time.perf_counter() - t0

    print(f'{nombre:<24} insertar: {t_insertar * 1000:8.1f} ms    buscar: {t_buscar * 1000:8.1f} ms')


def ins_dict(d, k, v):
    d[k] = v

def bus_dict(d, k):
    return d.get(k)


N = 20000
claves_bench = [''.join(random.choices(string.ascii_lowercase, k=8)) for _ in range(N)]
print(f'{N} claves de 8 letras\n')

benchmark('TablaHashChaining', TablaHashChaining,
          lambda t, k, v: t.insertar(k, v), lambda t, k: t.buscar(k), claves_bench)
benchmark('TablaHashOpenAddressing', TablaHashOpenAddressing,
          lambda t, k, v: t.insertar(k, v), lambda t, k: t.buscar(k), claves_bench)
benchmark('dict de Python', dict, ins_dict, bus_dict, claves_bench)
```

Salida — Python

```
20000 claves de 8 letras

TablaHashChaining        insertar:     61.5 ms    buscar:      4.3 ms
TablaHashOpenAddressing  insertar:     45.1 ms    buscar:      7.8 ms
dict de Python           insertar:      1.4 ms    buscar:      0.9 ms
```

Tabla 6 | Tablas propias vs `dict` (20 000 claves de 8 letras).

Estructura	Insertar (ms)	Buscar (ms)
`TablaHashChaining`	61.5	4.3
`TablaHashOpenAddressing`	45.1	7.8
`dict` de Python	1.4	0.9

Análisis. El `dict` es unas 30 a 45 veces más rápido al insertar y unas 5 a 9 veces más al buscar. Las tablas propias están escritas en Python puro: cada operación son llamadas a métodos, creación de objetos (nodos o tuplas) y bucles interpretados, mientras que `dict` está implementado en C, reutiliza el hash de los strings ya calculado y tiene un diseño de memoria compacto. Entre las dos propias, el encadenamiento busca más rápido (cadenas cortas por el rehash en $\alpha > 0.75$) y el sondeo lineal inserta más rápido (no crea nodos), lo que concuerda con la teoría. Los tiempos varían según la máquina, pero el orden de magnitud respecto a `dict` se mantiene.

ACTIVIDAD 5

C++ — Hash personalizado y unordered_map

Objetivo. *Implementar una función hash personalizada para std::string en C++17 y comparar su rendimiento con la hash por defecto.*

Función hash FNV-1a de 64 bits frente a `std::hash`, sobre 50 000 claves aleatorias de 8 letras (semilla 42). Se descarta una corrida de calentamiento y se promedian 10; el `checksum` evita que el compilador elimine las búsquedas.

Código — practica07 (C++17)

```cpp
// Actividad 5: función hash personalizada (FNV-1a) vs std::hash en un unordered_map
//
// Compilar:  g++ -std=c++17 -O2 -Wall -Wextra Practica_S07_TablaHash_ApellidoNombre.cpp -o practica07
// Ejecutar:  ./practica07

#include <chrono>
#include <cstdint>
#include <iostream>
#include <random>
#include <string>
#include <unordered_map>
#include <vector>

using Clock = std::chrono::steady_clock;

struct HashFNV
{
    std::size_t operator()(const std::string &s) const
    {
        std::uint64_t hash = 14695981039346656037ULL; // FNV offset basis
        for (unsigned char c : s)
        {
```

```
            hash ^= c;
            hash *= 1099511628211ULL; // FNV prime
        }
        return static_cast<std::size_t>(hash);
    }
};

struct Resultado
{
    double ms = 0; // tiempo promedio (insertar + buscar todas las claves)
    double load_factor = 0;
    std::size_t buckets = 0;    // cantidad de buckets de la tabla
    std::size_t vacios = 0;     // buckets sin ninguna clave
    std::size_t max_bucket = 0; // claves en el bucket más lleno (peor cadena)
    long long checksum = 0;     // evita que el compilador elimine las búsquedas
};

template <typename Hash>
Resultado benchmarkHash(const std::vector<std::string> &claves, int repeticiones)
{
    Resultado res;
    double total_ms = 0.0;

    for (int r = -1; r < repeticiones; r++)
    { // r = -1: calentamiento
        std::unordered_map<std::string, int, Hash> mapa;
        mapa.reserve(claves.size());

        auto t0 = Clock::now();
        for (std::size_t i = 0; i < claves.size(); i++)
        {
            mapa[claves[i]] = static_cast<int>(i);
        }
        long long suma = 0;
        for (const auto &k : claves)
        {
            suma += mapa.find(k)->second;
        }
        double ms = std::chrono::duration<double, std::milli>(Clock::now() - t0).count();

        if (r >= 0)
            total_ms += ms;

        if (r == repeticiones - 1)
        { // estadísticas de la última corrida
            res.load_factor = mapa.load_factor();
            res.buckets = mapa.bucket_count();
            for (std::size_t b = 0; b < mapa.bucket_count(); b++)
            {
                std::size_t tam = mapa.bucket_size(b);
                if (tam == 0)
                    res.vacios++;
                if (tam > res.max_bucket)
                    res.max_bucket = tam;
            }
            res.checksum = suma;
        }
    }

    res.ms = total_ms / repeticiones;
    return res;
}

void mostrar(const char *nombre, std::size_t n, const Resultado &r)
```

```cpp
{
    std::cout << nombre << "  n=" << n
              << "  tiempo=" << r.ms << " ms"
              << "  load_factor=" << r.load_factor << '\n'
              << "            buckets=" << r.buckets
              << "  vacios=" << r.vacios
              << "  max_en_bucket=" << r.max_bucket
              << "  checksum=" << r.checksum << "\n\n";
}

int main()
{
    // Generar 50 000 claves aleatorias de 8 letras minúsculas
    std::mt19937 rng(42);
    const std::string letras = "abcdefghijklmnopqrstuvwxyz";
    std::vector<std::string> claves;
    for (int i = 0; i < 50000; i++)
    {
        std::string s(8, ' ');
        for (char &c : s)
            c = letras[rng() % 26];
        claves.push_back(s);
    }

    const int REPETICIONES = 10;
    std::cout << "Benchmark con " << claves.size() << " claves, promedio de "
              << REPETICIONES << " corridas\n\n";

    Resultado r_std = benchmarkHash<std::hash<std::string>>(claves, REPETICIONES);
    Resultado r_fnv = benchmarkHash<HashFNV>(claves, REPETICIONES);

    mostrar("std::hash", claves.size(), r_std);
    mostrar("FNV-1a   ", claves.size(), r_fnv);

    double ratio = r_std.ms / r_fnv.ms;
    std::cout << "Ratio (std::hash / FNV-1a): " << ratio << "x\n";
    if (ratio > 1.0)
        std::cout << "-> FNV-1a fue mas rapida.\n";
    else
        std::cout << "-> std::hash fue mas rapida (o igual).\n";

    return 0;
}
```

Terminal — PowerShell

```
PS D:\ALGORITMO Y ESTRUCTURA\SEM7> g++ -std=c++17 -O2 -Wall -Wextra
    Practica_S07_TablaHash_ApellidoNombre.cpp -o practica07
PS D:\ALGORITMO Y ESTRUCTURA\SEM7> .\practica07
Benchmark con 50000 claves, promedio de 10 corridas

std::hash  n=50000  tiempo=3.98376 ms  load_factor=0.939832
           buckets=53201  vacios=20985  max_en_bucket=8  checksum=1249975000

FNV-1a     n=50000  tiempo=4.24391 ms  load_factor=0.939832
           buckets=53201  vacios=20662  max_en_bucket=8  checksum=1249975000

Ratio (std::hash / FNV-1a): 0.9387x
-> std::hash fue mas rapida (o igual).
PS D:\ALGORITMO Y ESTRUCTURA\SEM7>
```

Tabla 7 | FNV-1a vs `std::hash` (50 000 claves, promedio de 10 corridas).

	`std::hash`	FNV-1a
Tiempo promedio (insertar + buscar)	3.98 ms	4.24 ms
load_factor	0.939832	0.939832
Buckets	53201	53201
Buckets vacíos	20985 (39.4 %)	20662 (38.8 %)
Máx. claves en un bucket	8	8
Ratio `std::hash` / FNV-1a	0.9387×	

Análisis.

- **Distribución equivalente.** Con ese load_factor, un reparto aleatorio ideal deja vacíos cerca del 39 % de los buckets ($e^{-\alpha} \approx 0.391$, unos 20 800), y ambas funciones están a ese nivel. El bucket más lleno es el mismo (8): en calidad de dispersión no hay diferencia práctica.
- **Diferencia de velocidad pequeña.** FNV-1a tardó un 6.5 % más (ratio 0.9387×, o sea `std::hash` fue más rápida). La causa está en el costo de calcular el hash, no en las colisiones: FNV-1a procesa un byte por iteración con una multiplicación dependiente de la anterior, mientras que `std::hash` de libstdc++ se basa en una variante de MurmurHash que procesa los datos por bloques.
- **Cautela.** Con 50 000 claves el tiempo total es de unos 4 ms, así que una diferencia de 6 % es pequeña. Para afirmarla con más seguridad convendría aumentar el número de claves y de repeticiones. FNV-1a no justifica el cambio aquí, pero es una buena función simple y sin estado.
- **Checksum idéntico** (1249975000) en ambas: las dos estructuras guardaron y encontraron las mismas claves.

ACTIVIDAD 6

Python — Aplicación real: índice invertido con tabla hash

Objetivo. *Construir un índice invertido (usado en motores de búsqueda) que mapea cada palabra a la lista de documentos que la contienen, usando una tabla hash.*

Usa los imports de la celda inicial (`re`, `defaultdict`). Las palabras se limpian con una expresión regular y la consulta se procesa igual que los títulos. En `buscar` se trabaja sobre una copia *del primer conjunto, porque `&=` modifica el conjunto en el lugar y dañaría el índice.*

Código — índice invertido y búsqueda AND

```python
# Catálogo de la Biblioteca Central UNA-PUNO
DOCUMENTOS = {
    1: 'Algoritmos y estructuras de datos en Python para ingeniería',
    2: 'Análisis de complejidad computacional y algoritmos recursivos',
    3: 'Estructuras de datos lineales: listas pilas y colas',
    4: 'Tablas hash y funciones de dispersión para bases de datos',
    5: 'Árboles binarios de búsqueda y algoritmos de recorrido',
    6: 'Grafos y algoritmos de caminos mínimos en redes viales',
    7: 'Fundamentos de programación en C++ para sistemas embebidos',
}

STOP_WORDS = {'y', 'de', 'en', 'para', 'el', 'la', 'los', 'las', 'a', 'e', 'o', 'un', 'una'}


def tokenizar(texto: str) -> list:
```

```python
    """Pasa a minúsculas y separa en palabras (sin signos de puntuación)."""
    return re.findall(r'\w+', texto.lower())


def construir_indice_invertido(docs: dict) -> dict:
    """
    Construye un índice invertido: {palabra: {doc_id, ...}}
    Complejidad: O(N), donde N = número total de palabras en todos los documentos.
    """
    indice = defaultdict(set)
    for doc_id, titulo in docs.items():
        for palabra in tokenizar(titulo):
            if palabra not in STOP_WORDS and len(palabra) > 2:
                indice[palabra].add(doc_id)
    return dict(indice)


def buscar(indice: dict, consulta: str) -> list:
    """Búsqueda multi-término: AND de todos los términos."""
    palabras = tokenizar(consulta)
    if not palabras:
        return []
    resultados = set(indice.get(palabras[0], set()))    # copia: no se toca el índice
    for p in palabras[1:]:
        resultados &= indice.get(p, set())
    return sorted(resultados)

indice = construir_indice_invertido(DOCUMENTOS)
print(f'Vocabulario: {len(indice)} términos únicos')
print(f'Documentos promedio por término: {sum(len(v) for v in indice.values()) / len(indice):.2f}')

consultas = ['algoritmos', 'estructuras datos', 'tablas hash', 'árboles grafos']
for q in consultas:
    ids = buscar(indice, q)
    titulos = [DOCUMENTOS[i] for i in ids]
    print(f'\nConsulta: {q!r}')
    print(f'  Resultados ({len(ids)}): {titulos}')

# Comprobación: las consultas no modificaron el índice
assert indice == construir_indice_invertido(DOCUMENTOS)
print('\nÍndice intacto tras las consultas ✓')
```

Salida — Python

```
Vocabulario: 31 términos únicos
Documentos promedio por término: 1.19

Consulta: 'algoritmos'
  Resultados (4): ['Algoritmos y estructuras de datos en Python para ingeniería', 'Análisis de
    complejidad computacional y algoritmos recursivos', 'Árboles binarios de búsqueda y algoritmos de
    recorrido', 'Grafos y algoritmos de caminos mínimos en redes viales']

Consulta: 'estructuras datos'
  Resultados (2): ['Algoritmos y estructuras de datos en Python para ingeniería', 'Estructuras de datos
    lineales: listas pilas y colas']

Consulta: 'tablas hash'
  Resultados (1): ['Tablas hash y funciones de dispersión para bases de datos']

Consulta: 'árboles grafos'
  Resultados (0): []
```

```
índice intacto tras las consultas ✓
```

Lectura del resultado. El vocabulario tiene 31 términos únicos y cada término aparece en promedio en 1.19 documentos. La consulta `'algoritmos'` devuelve 4 títulos; `'estructuras datos'` devuelve 2 (los que contienen ambas palabras); `'tablas hash'` devuelve 1; y `'árboles grafos'` devuelve 0 porque, con búsqueda AND, ningún título contiene las dos palabras a la vez. La comprobación final confirma que las consultas no alteraron el índice.

III. PREGUNTAS DE ANÁLISIS

PREGUNTA 1

En la Actividad 3 (Open Addressing), ¿por qué usar DELETED en lugar de VACIA al eliminar es crítico para la corrección del algoritmo? Construya un ejemplo concreto de 5 inserciones donde usar VACIA produciría un falso negativo en la búsqueda.

Respuesta. **Idea clave.** `buscar` se detiene en la **primera celda `VACIA`**, porque una celda nunca usada significa «aquí termina la cadena de sondeo, la clave no existe». Si al eliminar se deja `VACIA`, se cortan las cadenas de los elementos que pasaron por esa celda al insertarse.

Ejemplo con 5 inserciones ($m = 10$, $h(k) = k$ mod 10, sondeo lineal). Las claves 10, 20, 30, 40 y 50 caen todas en la celda 0:

Paso	Operación	Estado de la tabla (celdas 0–5)
1	`insertar(10)`	`[10, _, _, _, _, _]`
2	`insertar(20)`	`[10, 20, _, _, _, _]`
3	`insertar(30)`	`[10, 20, 30, _, _, _]`
4	`insertar(40)`	`[10, 20, 30, 40, _, _]`
5	`insertar(50)`	`[10, 20, 30, 40, 50, _]`
6	`eliminar(20)` **con VACIA**	`[10, _, 30, 40, 50, _]`

Ahora `buscar(30)`: celda 0 ($10 \neq 30$) → la celda 1 está `VACIA` → el algoritmo concluye «no existe» y devuelve `None`. Es un **falso negativo**: el 30 sigue guardado en la celda 2. Lo mismo pasa con `buscar(40)` y `buscar(50)`.

Con `DELETED` en la celda 1, la búsqueda ve «celda eliminada, sigo sondeando», llega a la celda 2 y encuentra el 30 (es el caso de la «Prueba 2» de la Actividad 3).

Dos detalles importantes de la solución:

- Al **insertar**, se puede reutilizar una celda `DELETED`, pero solo después de recorrer la cadena hasta una `VACIA` para asegurarse de que la clave no existe más adelante. Si no, se podría duplicar una clave.
- Los `DELETED` ocupan espacio: se cuentan en el factor de carga y se eliminan en el siguiente rehash. Si no se contaran, la tabla podría llenarse de marcadores y las búsquedas fallidas recorrerían casi toda la tabla.

PREGUNTA 2

El miniestudio (Actividad 4) muestra que el sondeo lineal se degrada mucho más rápido que el encadenamiento cuando $\alpha > 0.75$. ¿Por qué ocurre esto en términos del fenómeno de «clustering primario»? ¿El sondeo cuadrático mejoraría este comportamiento?

Respuesta. **Por qué el sondeo lineal se degrada más rápido.** En el sondeo lineal las celdas ocupadas se agrupan en **bloques contiguos** (clusters). Cuando una clave cae dentro de un bloque, tiene que avanzar hasta el final y ocupa la celda siguiente, con lo que **el bloque crece**. Cuanto más largo es un bloque, más probable es que la próxima clave caiga en él (la probabilidad es proporcional a su longitud), y además dos

bloques separados por una sola celda libre se fusionan al llenarse. Es un efecto de «los grandes crecen más»: a medida que α sube, aparecen bloques largos y los sondeos crecen de forma no lineal.

Los costos esperados lo muestran (búsqueda fallida):

alpha	Sondeo lineal $\frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right)$	Encadenamiento $\approx 1+\alpha$
0.50	2.5	1.50
0.75	8.5	1.75
0.90	50.5	1.90

Concuerda con el miniestudio: el sondeo lineal se separa del encadenamiento justo cuando α pasa de 0.5 a 0.75 (de 2.3× a 4.8×) y se dispara en $\alpha = 1$ (48×).

¿Mejoraría el sondeo cuadrático? Sí, en parte. Con la secuencia $h(k) + c_1 j + c_2 j^2$, dos claves que caen en el mismo bloque dejan de seguir el mismo camino, así que **desaparece el clustering primario**. Pero:

- Queda el **clustering secundario**: claves con el mismo hash inicial siguen exactamente la misma secuencia de sondeo.
- Para garantizar que se encuentre una celda libre hay que elegir con cuidado m y las constantes (por ejemplo, m primo y $\alpha < 0.5$ con $h + j^2$).
- Sigue sin poder superar $\alpha = 1$ y pierde la localidad de caché del sondeo lineal.
- La opción más cercana al ideal es el **doble hashing**, donde el paso depende de una segunda función hash de la clave.

> PREGUNTA 3
>
> *En el índice invertido (Actividad 6), ¿cuál es la complejidad de la operación buscar() en términos del número de palabras en la consulta (q) y del número de documentos que contienen cada palabra (k)? ¿Cómo se modifica si se quiere búsqueda OR en lugar de AND?*

Respuesta. Sea q el número de palabras de la consulta y k el máximo de documentos que contienen una palabra (tamaño de la lista de resultados de un término).

Búsqueda AND (la implementada):

Paso	Costo
Separar la consulta en palabras	$O(\text{longitud de la consulta}) \approx O(q)$
q consultas al diccionario (`indice.get`)	$O(1)$ promedio cada una $\rightarrow O(q)$
Copiar el primer conjunto	$O(k)$
$q-1$ intersecciones `&=`	cada una cuesta $O(\min(\lvert\text{resultados}\rvert, k_i)) \le O(k) \rightarrow O(q \cdot k)$
Ordenar el resultado final	$O(r \log r)$, con $r \le k$

Total: $O(q \cdot k)$ (más el $O(r \log r)$ de ordenar, con $r \le k$). Se puede mejorar procesando primero las palabras con menos documentos y cortando en cuanto el resultado quede vacío.

Si se quiere búsqueda OR. Ahora hay que **unir** los conjuntos: se parte de un conjunto vacío nuevo (nunca del conjunto del índice, para no modificarlo) y se une cada término:

Código — búsqueda OR

```python
def buscar_or(indice: dict, consulta: str) -> list:
    """Búsqueda multi-término: OR de todos los términos."""
    resultados = set()                      # conjunto nuevo
    for p in tokenizar(consulta):
        resultados |= indice.get(p, set())
    return sorted(resultados)
```

- La unión cuesta $O(\sum k_i) \leq O(q \cdot k)$: el mismo orden asintótico que AND, pero **sin posibilidad de corte temprano** (hay que leer todas las listas).
- El resultado puede ser mucho mayor (hasta $q \cdot k$ documentos, acotado por el total de documentos), así que el ordenamiento final pasa a $O(R \log R)$, con R el tamaño de la unión.

PREGUNTA 4

Python dict preserva el orden de inserción desde Python 3.7. ¿Implica esto que el dict internamente es una lista ordenada? Investigue brevemente cómo logra Python preservar el orden sin sacrificar el acceso $O(1)$.

Respuesta. No. Conserva el **orden de inserción** (no el orden de las claves): si se inserta `'b'` y luego `'a'`, el recorrido da `'b'`, `'a'`. Eso es lo que se garantiza desde Python 3.7 (en CPython 3.6 ya ocurría, pero como detalle de implementación). Y el acceso sigue siendo $O(1)$ en promedio.

Cómo lo logra: el «dict compacto». En lugar de una sola tabla dispersa, CPython usa **dos arreglos**:

1. **`indices`:** una tabla hash pequeña y dispersa (tamaño potencia de 2) que no guarda las claves, solo **números de posición** (enteros chicos) hacia el otro arreglo, o un valor especial si la celda está vacía o eliminada.
2. **`entries`:** un arreglo **denso** donde cada nueva entrada (hash, clave, valor) se **agrega al final**, en orden de inserción.

Ejemplo ilustrativo, tras insertar `'a'`, `'b'`, `'c'` (las posiciones en `indices` son solo de ejemplo):

Estructura interna (ilustrativa)

```
indices : [ -1, 1, -1, 2, -1, 0, -1, -1 ]     (tabla dispersa: guarda posiciones)
entries : [ ('a', 1), ('b', 2), ('c', 3) ]    (arreglo denso, en orden de inserción)
```

- **Buscar:** se calcula el hash de la clave → se mira esa celda en `indices` → se lee la posición → se compara la clave en `entries`. Un par de accesos, $O(1)$ promedio.
- **Insertar:** se agrega al final de `entries` y se anota su posición en `indices`. Es $O(1)$ amortizado.
- **Recorrer:** se camina `entries` de principio a fin: ya está en orden de inserción, y sin saltar por celdas vacías.
- **Eliminar:** la entrada queda marcada como borrada y la celda de `indices` recibe un marcador especial. Es la misma idea del `DELETED` de la Pregunta 1. Los huecos se eliminan cuando la tabla se redimensiona.
- **Colisiones:** `indices` usa direccionamiento abierto, pero **no con sondeo lineal**: el paso mezcla bits del hash para evitar el clustering primario. Se redimensiona cuando se llena aproximadamente dos tercios.

Ventajas. Ahorra memoria respecto al diseño anterior (se habla de entre 20 % y 25 %) y hace más rápida la iteración. El orden es una consecuencia de usar `entries` como arreglo que solo crece. Un detalle: reasignar el valor de una clave existente no cambia su posición, pero eliminar una clave y volver a insertarla la lleva al final.

Fuentes sugeridas: la documentación de Python sobre `dict` y la propuesta de Raymond Hettinger de 2012 en la lista python-dev, «More compact dictionaries with faster iteration».